

Al-Aqsa University

Faculty of Computers and IT

Engineering and Networks Dept.



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات

Operating Systems for Intelligent Systems (ENGS2306)

Lecture 7: Advanced Bash Scripting

Lecturer: Eng. Mohammed E. Alhams

Semester: Second Semester 2024/2025

Lecture 7: Advanced Bash Scripting

Today's Objectives:

1. Organize code into reusable **functions**.
 2. Manage lists of data using **arrays**.
 3. Implement robust **error handling** to make scripts more reliable.
 4. Combine these concepts to write a modular, professional-grade script.
-

Recap: The Limits of Basic Scripts

In **Lecture 6**, we learned the fundamentals of scripting:

- Variables to store data.
- if statements for decisions.
- for and while loops for repetition.

The Problem: As our tasks get more complex, our scripts can become long, hard to read, and difficult to maintain. What if we need to perform the same 10-line task multiple times in one script? Copy-pasting code is inefficient and a recipe for bugs.

The Solution: We need to make our scripts **modular**, **scalable**, and **robust**. Today, we learn how.

Modular Code with Functions

A **function** is a named block of code that you can call multiple times from different parts of your script. This follows the **DRY (Don't Repeat Yourself)** principle.

Basic Syntax:

```
function_name() {  
    # Block of code goes here  
    echo "This is my first function!"  
}  
  
# --- To run the function, just call its name ---  
function_name
```

Why use functions?

- **Readability:** The main part of your script becomes a clean sequence of function calls.
- **Reusability:** Write the logic once, use it everywhere.
- **Maintainability:** If you need to fix a bug, you only fix it in one place—the function.

Functions with Parameters

You can pass data *into* a function using **positional parameters**. Inside the function, they are accessed as \$1 (the first argument), \$2 (the second), and so on.

Example:

```
#!/bin/bash
# A function that greets a specific user.

greet_user() {
    local user_name=$1 # Good practice to use 'local' for variables inside
    functions
    echo "Hello, $user_name! Welcome."
}

# Call the function with different parameters
greet_user "Ahmed"
greet_user "Fatima"
```

- \$@ is a special variable that represents all arguments passed to the function.

Getting Data Out of Functions

Bash functions don't "return" data in the way languages like Python or C do. They have two main ways of providing output:

1. Exit Status (for success/failure):

A function returns an **exit code** (0 for success, 1-255 for failure). You can check this with \$?.

```
return 0 # Success
```

```
return 1 # Failure
```

2. Standard Output (for data):

To "return" a value or string, you echo the data. The calling part of the script captures this output using **command substitution**.

Example:

```
get_timestamp() {
    date "+%Y-%m-%d %H:%M:%S"
}

# Capture the echoed output in a variable
log_time=$(get_timestamp)
echo "Event occurred at: $log_time"
```

Managing Lists with Arrays

An **array** is a special variable that can hold a list of multiple values. This is perfect for managing lists of devices, users, or files.

Syntax:

```
# Declare an array
iot_devices=("192.168.1.50" "192.168.1.51" "sensor-gateway")

# Access a single element (indexing starts at 0)
echo "First device: ${iot_devices[0]}"

# Access all elements
echo "All devices: ${iot_devices[@]}"

# Get the number of elements
echo "Total devices: ${#iot_devices[@]}"
```

Important: When expanding an array, always quote it ("\${my_array[@]}") to correctly handle items with spaces.

Looping Through Arrays

The most common use of an array is to iterate through it with a for loop.

Syntax:

```
my_array=("item1" "item2" "item3")

for item in "${my_array[@]}; do
    echo "Processing: $item"
done
```

Intelligent Systems Example: A script to update software on multiple Raspberry Pi devices.

```
#!/bin/bash
PI_HOSTS=("pi-zero1" "pi-compute4" "pi-gateway")

for host in "${PI_HOSTS[@]}; do
    echo "--- Updating $host ---"
    ssh "pi@$host" "sudo apt update && sudo apt upgrade -y"
    echo "--- Finished $host ---"
done
```

Robust Error Handling with set -e

What happens if a command in your script fails? By default, the script just continues running, which can lead to disastrous results (e.g., trying to write to a backup directory that failed to be created).

set -e is a command that tells the shell to **exit immediately** if any command fails (returns a non-zero exit code).

Place this near the top of your scripts to make them safer:

```
#!/bin/bash
set -e

# This command will fail if the directory already exists without -p
mkdir /nonexistent_root_dir/my_app
# Because of 'set -e', the script will stop here.
echo "This line will never be reached."
```

This is a simple but powerful way to ensure your automation stops when something goes wrong.

Practical Scenario: System Service Health Check

Let's write a script that uses functions and arrays to check the status of critical system services.

1. **Create the script file:** `touch health_check.sh && chmod +x health_check.sh`
 2. **Add the basics:** `#!/bin/bash` and `set -e`.
 3. **Define an array** of services to check: `SERVICES=("ssh" "cron" "ufw")`
 4. **Create a function** `check_service()` that takes one argument (the service name).
 5. **Inside the function**, use `systemctl is-active --quiet "$1"` to check the service. The `--quiet` flag makes it only return an exit code (0 if active, non-zero if inactive/failed).
 6. **Inside the function**, use an if statement to check the exit code (`$?`) of the `systemctl` command. Print a success or failure message.
 7. **In the main part of the script**, print a header like `--- System Health Report ---`.
 8. **Use a for loop** to iterate through your `SERVICES` array.
 9. **Inside the loop**, call your `check_service()` function for each service.
 10. **Run the script:** `./health_check.sh`.
-

The Idea Behind the Assignment

Why are these advanced concepts critical for Intelligent Systems?

- **Modularity & Complexity:** A modern intelligent system (like a robot or a smart home hub) is not one program; it's dozens of services working together. A management script must be modular. You can have functions like `check_camera_feed()`, `restart_wifi()`, `read_all_sensors()`. The main script logic remains clean and understandable.
 - **Scalability:** Imagine deploying a fleet of 100 IoT sensors. You don't write 100 different scripts. You write one script that reads a list of sensor IPs from an array (or a file) and loops through them, performing the same configuration or update task on each.
 - **Reliability & Safety:** For a system that operates autonomously (like a drone or an industrial controller), unattended failure is not an option. Robust error handling (`set -e`, checking exit codes) ensures that if a critical component fails (e.g., a motor controller doesn't respond), the script stops safely instead of proceeding with a dangerous, partially-failed operation.
-

Assignment 6: Modular System Health Reporter (12%)

Objective: To write a modular and user-friendly script using functions, arrays, and conditional logic to generate a system health report.

Tasks:

1. Create a script named `system_reporter.sh`.
2. At the top, include `#!/bin/bash` and `set -e`.
3. Define an array containing the names of at least three services you want to check (e.g., `ssh`, `cron`, `ufw`, `cups`).
4. Create a function named `check_service_status()`. This function will take one argument: the name of a service.
5. Inside the function, it should check if the service is active using `systemctl is-active --quiet "$1"`.
6. Based on the exit code (`$?`), the function should print the status in a user-friendly, colored format. For example:
 - If active: `echo -e " [ OK ] $1 is active."`
 - If inactive/failed: `echo -e " [FAIL] $1 is not active."`
7. In the main part of your script:
 - Print a header, e.g., "System Health Report for `$(hostname)` as of `$(date)`".

- Use a for loop to iterate through your array of services.
 - Inside the loop, call your `check_service_status` function for each service.
 - Print a footer, e.g., "--- End of Report ---".
-

Assignment Submission Instructions

- **File Name:** Name your submission file exactly as follows:
studentnum-studentname-Assignment6.pdf
 - *Example:* 13012070xx-MohammedEmadAlhams-Assignment6.pdf
 - **Terminal Prompt:** Your name must appear in the terminal prompt for your screenshots.
 - **File Format:** Your submission must be a **single PDF file** containing:
 - The complete code of your `system_reporter.sh` script (copy-paste as text).
 - A screenshot showing the output of your script running successfully.
 - A brief report (1-2 paragraphs) explaining the primary benefit of using a function (`check_service_status`) in your script compared to putting all the checking logic directly inside the for loop.
 - **Academic Integrity:**
 - **Warning:** Identical submissions will be considered cheating and will result in a grade of **ZERO**.
 - **Warning:** AI-generated solutions or reports will result in a grade of **ZERO**.
-

Next Steps & Resources

Practice This Week:

- Modify your health reporter. Can you create another function that checks disk space (`df -h`) and add it to the report?
- Try passing two arguments to a function. For example, a function that creates a directory (\$1) and sets its permissions (\$2).

Next Week: Lecture 8 - Linux Networking Basics

- We will shift gears from scripting to networking. We'll learn the essential commands to configure, manage, and troubleshoot network connections on Linux, a vital skill for any connected intelligent system.

Gaza Connectivity Tips:

- Download this lecture and practice offline. Your VM doesn't need an internet connection to write and test Bash scripts.
- Focus on understanding the logic, not just copying the code.

Resources:

- Bash Functions and Local Variables: <https://tldp.org/LDP/abs/html/functions.html>
- Bash Arrays: <https://opensource.com/article/18/5/you-dont-know-bash-intro-bash-arrays>
- Error Handling (set -e): https://vaneyckt.io/posts/safer_bash_scripts_with_set_euxo_pipefail/
- Course Materials on Moodle